Reminders and announcements

- During lectures you can submit questions to: vevox.app ID: 107-018-952
- You can find answers to questions in the COMP1870 Teams Channel (link in Minerva) (along with other discussion).
- Maths Lunch Hours, Mondays and Wednesdays, 1.00pm - 2.00pm. Bragg GR.18.
- My student hour: Tuesdays 1.00pm - 2.00pm, Bragg building foyer.

Revision

- If p and q are integers, then $p|q$ means that $q = kp$ for some $k \in \mathbb{Z}$. We say that p **divides** q, or p is a **factor** of q, or q is a **multiple** of p.
- (It is natural to say that p divides q if $\frac{q}{p}$ is an integer, but this definition creates problems with division by zero.)
- If p is any integer, then $p|0$, $1|p$, and $p|p$.
- If $p \neq 0$, then $0 \nmid p$.
- If $p|a$, then $p|(ab)$, for any integer b.
- If $p|a$ and $p|b$, then $p|(a + b)$.

Properties of divisibility

Proposition 9.3. Let A be a set of positive integers. Consider the following relation R on the elements of A. If a and b are positive integers then we declare (a, b) to be in R if and only if a divides b. Then R is a partial order.

Properties of divisibility

Proposition 9.3. Let A be a set of positive integers. Consider the following relation R on the elements of A. If a and b are positive integers then we declare (a, b) to be in R if and only if a divides b. Then R is a partial order.

Number representations

When we represent an integer with the expression 41025, we mean the integer

$$4 \cdot 10^4 + 1 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0.$$

This is **base-10** (**decimal**) representation.

We can represent integers using other **bases**. If $b > 1$ is an integer, then the expression $(a_n a_{n-1} \cdots a_1 a_0)_b$ means

$$a_n \cdot b^n + a_{n-1} b^{n-1} + \cdots + a_1 b^1 + a_0 b^0.$$

where each a_i is a digit between 0 and $b - 1$.

Number representations

Common choices of bases are:

- 2 (**binary** representation),
- 8 (**octal** representation),
- 10 (**decimal** representation),
- 16 (**hexadecimal** representation).

In hexadecimal representation, we need digits for all of the numbers $0, 1, 2, \ldots, 15$.

A	B	C	D	E	F
10	11	12	13	14	15

Number representations

In the representation $(a_n a_{n-1} \cdots a_1 a_0)_b$, we say a_n is the **leading digit**. We always assume the leading digit is non-zero. For example, we would write $(401)_{10}$ rather than $(0401)_{10}$.

Proposition 9.4. Let b be an integer greater than 1 and let m be a non-negative integer. Then m has exactly one base-b representation.

Number representations

Number representations

Logarithms

Proposition 9.6. Let b be an integer greater than 1. If m is a non-negative integer, and n is the smallest integer such that $b^n > m$, then it takes exactly n digits to represent m in base b.

Logarithms

Recall that a **logarithm** is an inverse of an exponential function.

$$\log_b x = y \text{ means } b^y = x.$$

We say that b is the **base** of the logarithmic function $\log_b$.

Proposition 9.8. Let b be an integer greater than 1. If m is a positive integer, then the number of digits in the base-b representation of m is $\lfloor \log_b m \rfloor + 1$.

Logarithms

We can convert between logarithms with different bases by multiplying by a constant:

Exercise. Let a and b be integers greater than 1. Then

$$\log_b m = \frac{1}{\log_a b}(\log_a m).$$

All logarithms are the same, up to multiplying by a constant. For this reason, we often say that the number of digits needed to represent m is $\log m$, without specifying the base.

Polynomial-time algorithms

We want algorithms with running times that look like this:

$$(\text{input size})^2 \text{ or } (\text{input size})^3 \text{ or, generally } (\text{input size})^c$$

(where c is a constant number).

These are called **polynomial-time algorithms**.

We do not want algorithms with running times that look like this:

$$2^{(\text{input size})} \text{ or } (\text{input size})!$$

Polynomial-time algorithms for arithmetic

Question. What is the input size of the integer m?

It is not m, since we don't need m digits to represent m.

Important point. The input size of the integer m is $\log m$.

We need algorithms which will carry out arithmetic operations with a running time of the form $(\log m)^c$.

If an algorithm is polynomial-time with respect to representation in one base, it is polynomial-time with respect to representations in any base.

Adding integers

The addition and multiplication algorithms we learn in school are polynomial-time algorithms.

Example. Add $(6034)_8$ and $(127)_8$.

Multiplying integers

Example. Multiply $(704)_8$ by $(25)_8$.

Exponentiation

Question How do we calculate something like $(11)_2^{(1010)_2}$?

Note that we cannot multiply together $(1010)_2 = (10)_{10}$ copies of $(11)_2 = (3)_{10}$, because this does not lead to a polynomial-time algorithm!

Remember that the input size of $(1010)_2$ is not 10. It is 4, because the number can be represented with 4 binary digits.

The method of repeated squaring

Question How do we calculate $x^{(y_n y_{n-1} \cdots y_2 y_1 y_0)_2}$?

$$
\begin{aligned}
& x^{(y_n y_{n-1} \cdots y_2 y_1 y_0)_2} \\
&= x^{2^n y_n + 2^{n-1} y_{n-1} + \cdots + 4y_2 + 2y_1 + y_0} \\
&= x^{2^n y_n} x^{2^{n-1} y_{n-1}} \cdots x^{4y_2} x^{2y_1} x^{y_0} \\
&= (x^{2^n})^{y_n} \cdots (x^4)^{y_2} (x^2)^{y_1} (x)^{y_0}
\end{aligned}
$$

The sequence $x, x^2, x^4, x^8, \ldots, x^{2^n}$ can be obtained by repeatedly squaring.

We keep a running product, z. When we process x^{2^i}, we check to see if $y_i = 1$. If so, we multiply z by x^{2^i}. If $y_i = 0$, we leave z unchanged.

Repeated squaring algorithm

Input: Base-2 representations of x and $y = (y_t y_{t-1} \cdots y_1 y_0)_2$.
Output: A representation of $z = x^y$.

$z \leftarrow 1$
$p \leftarrow x$
for $i = 0$ to t **do**
    if $y_i = 1$ **then**
        $z \leftarrow z \cdot p$
    end if
    $p \leftarrow p^2$
end for

The method of repeated squaring

Question How do we calculate $(11)_2^{(1010)_2}$?

i	y_i	z	p
0	0		
1	1		
2	0		
3	1		

Quotients and remainders

Theorem 9.9. Let m and n be positive integers. There are unique integers q and r such that $m = qn + r$, and $0 \leq r < n$.

We say q is the **quotient** and r is the **remainder** when we divide m by n.

Quotients and remainders

Theorem 9.9. Let m and n be positive integers. There are unique integers q and r such that $m = qn + r$, and $0 \leq r < n$.

Quotients and remainders

Theorem 9.9. Let m and n be positive integers. There are unique integers q and r such that $m = qn + r$, and $0 \leq r < n$.

Quotients and remainders

Question. How do we find the quotient and remainder if we divide 2710 by 26?

We could repeatedly subtract 26 from 2710 and count the number of loops it takes to reduce to a number that is less than 26.

This does not lead to a polynomial-time algorithm!

Quotients and remainders

Question. How do we find the quotient and remainder if we divide 2710 by 26?

Calculating quotient and remainder

Input: Base-b representations of $m = (m_s m_{s-1} \cdots m_1 m_0)_b$ and $n = (n_t n_{t-1} \cdots n_1 n_0)_b$ $(n \leq m)$.

Output: q and r such that $m = qn + r$ and $0 \leq r < n$.

$r \leftarrow m$
$i \leftarrow s - t$
$q \leftarrow 0$
while $r \geq n$ **do**
    while $r \geq nb^i$ **do**
        $r \leftarrow r - nb^i$
        $q \leftarrow q + b^i$
    end while
    if $i > 0$ **then**
        $i \leftarrow i - 1$
    end if
end while